



Rajshahi University of Engineering & Technology
Department of Computer Science & Engineering

Multi-Label Image Classification on KIIT-MiTA Dataset

A Comprehensive Study of Deep Learning Architectures with BattleNet

Submitted By
Abu Bakar Siddik
Student ID: 2410103036

Submitted To
Dr. Md. Rabiul Islam
Professor, Department of Computer Science & Engineering

Course: Deep Learning — Course No.: CSE-6312
March 2026

Keywords: *Multi-label Classification, Deep Learning, Vision Transformers, BattleNet, KIIT-MiTA, Transfer Learning, Label Co-occurrence*

Code Repository: github.com/abubakarsiddik31/battle-vision

Abstract

Multi-label image classification presents unique challenges compared to traditional single-label classification, as images may contain multiple objects simultaneously. This study presents a comprehensive evaluation of modern deep learning architectures for multi-label classification on the **KIIT-MiTA dataset**, which contains military-themed images across seven classes: Artillery, Missile, Radar, M. Rocket Launcher, Soldier, Tank, and Vehicle. We systematically evaluate eleven architectures spanning three families — Classic CNNs (VGG16), Modern CNNs (ResNet, DenseNet, MobileNet, EfficientNet, ConvNeXt), and Vision Transformers (ViT, Swin-T) — through **39 experiments across seven optimization phases**. Building on these findings, we propose **BattleNet**, a novel architecture that combines the Swin-T backbone with a specialized classification head incorporating Squeeze-Excitation recalibration, a residual MLP transformation, and a learnable label co-occurrence matrix. BattleNet achieves a **Micro F1 of 0.8178** and **Macro F1 of 0.8319**, outperforming the best Swin-T baseline (Micro F1: 0.8121) on identical hardware and training configuration. Our analysis reveals that: (1) Vision Transformers outperform CNNs on this dataset; (2) weight decay $1e-3$ with cosine annealing is the optimal training configuration; (3) label co-occurrence modeling provides consistent gains across military object classes; and (4) class imbalance techniques improve minority class performance at the cost of overall metrics.

1. Introduction

1.1 Background and Motivation

The world is experiencing a dramatic and alarming rise in armed conflicts. Since 2022, the global security landscape has deteriorated significantly: the Russia-Ukraine war — the largest armed conflict in Europe since World War II — has demonstrated the pivotal role of advanced military technologies, including missile systems, armored vehicles, drones, and radar installations, in modern warfare. The Israel-Gaza conflict, intensifying from October 2023, further highlighted the strategic importance of precision-guided munitions and aerial surveillance. In parallel, conflicts in Sudan, Myanmar, the Sahel region of Africa, and elsewhere have stretched to involve dozens of simultaneous active war zones worldwide. According to data from the Stockholm International Peace Research Institute (SIPRI), the number of state-based armed conflicts in 2023-2024 reached the highest level since the end of the Cold War, displacing millions and resulting in catastrophic humanitarian consequences.

These conflicts have underscored a fundamental truth: intelligent automated military asset recognition is no longer a luxury but a strategic necessity. The ability to rapidly and accurately identify battlefield assets — artillery positions, missile launchers, radar arrays, armored vehicles, and troop formations — from aerial and ground-based imagery provides a decisive advantage in situational awareness, threat assessment, and tactical decision-making. Historically, such analysis has relied on human intelligence analysts, but the sheer volume of imagery data generated by modern reconnaissance systems far exceeds human processing capacity.

1.2 The Bangladeshi Perspective

Bangladesh presents a unique and compelling case for investment in military AI capabilities. Despite having approximately 165,000 active armed forces personnel, Bangladesh's defense budget stands at roughly 1.2% of GDP — significantly lower than regional neighbors such as India (with a defense budget exceeding USD 83 billion) and Pakistan (USD 10+ billion). This resource disparity means Bangladesh cannot rely on volume or hardware parity to ensure national security. Instead, it must leverage intelligent, cost-effective technologies to maximize its defensive capabilities.

Bangladesh's strategic position — bordered almost entirely by India, with a shared border with Myanmar where an active civil war has produced cross-border incursions and a massive refugee crisis — makes situational awareness and border monitoring critically important. The Bangladesh Armed Forces' "Forces Goal 2030" modernization initiative explicitly recognizes the need to incorporate emerging technologies. However, indigenous defense AI research and development remains nascent, with the country heavily dependent on imported military systems from China and Russia, limiting both affordability and customization.

This research is directly motivated by this gap. By developing a high-performance, open-source AI system for military object recognition — built on freely available deep learning frameworks and publicly accessible datasets — we aim to contribute to

Bangladesh's defense technology ecosystem. An automated multi-label military asset classification system can serve as a foundation for intelligent surveillance platforms, border monitoring applications, and reconnaissance analysis tools that are both affordable and adaptable to the specific needs of the Bangladesh Armed Forces.

1.3 Technical Background

Multi-label image classification is a fundamental computer vision problem where an image may be associated with multiple labels simultaneously. Unlike single-label classification, where each image belongs to exactly one category, multi-label classification requires models to identify and predict multiple co-occurring objects. The KIIT-MiTA dataset presents this challenge in the military domain, containing images across seven distinct classes that frequently co-occur — soldiers alongside vehicles, missiles near launchers, radar systems in fortified positions. This co-occurrence structure reflects real-world military scenarios and makes the task both technically demanding and practically significant.

1.4 Research Objectives

The primary objectives of this study are:

1. **Architecture Evaluation:** Systematically compare representative architectures from Classic CNNs, Modern CNNs, and Vision Transformers on the KIIT-MiTA dataset.
2. **Hyperparameter Optimization:** Investigate the impact of regularization, optimization strategies, and training configurations on model performance.
3. **Novel Architecture Proposal:** Design and evaluate BattleNet — a specialized architecture augmenting Swin-T with a task-specific classification head tailored for military multi-label classification.
4. **Label Co-occurrence Modeling:** Explore learnable label dependency structures to exploit co-occurrence patterns inherent in military imagery.

1.5 Contributions

- A comprehensive experimental evaluation of 11 architectures across 39 experiments on the KIIT-MiTA dataset
- Identification of Swin-T with weight decay $1e-3$ and cosine annealing as the optimal baseline (Micro F1: 0.8121)
- Proposal of BattleNet achieving Micro F1 of 0.8178 (+0.57%) and Macro F1 of 0.8319 (+0.78%) over the Swin-T baseline
- Demonstration that label co-occurrence modeling with SE recalibration provides consistent per-class improvements, particularly for Missile (+4.1%) and M. Rocket Launcher (+5.3%)
- Open-source code available at: github.com/abubakarsiddik31/battle-vision

2. Related Work

2.1 Convolutional Neural Networks

Convolutional Neural Networks (CNNs) have dominated image classification since AlexNet's 2012 breakthrough. Key architectures include:

- VGG Networks (Simonyan & Zisserman, 2014): Deep networks with small 3×3 filters
- ResNet (He et al., 2016): Residual connections enabling very deep network training
- DenseNet (Huang et al., 2017): Dense connectivity improving feature reuse and gradient flow
- EfficientNet (Tan & Le, 2019): Compound scaling for efficiency
- ConvNeXt (Liu et al., 2022): Modern CNN designs inspired by Vision Transformers

2.2 Vision Transformers

Vision Transformers (ViT) (Dosovitskiy et al., 2020) revolutionized image classification by applying transformer architectures to computer vision:

- Vision Transformer (ViT): Pure transformer architecture using patch embeddings
- Swin Transformer (Liu et al., 2021): Hierarchical transformer with shifted windows for better scalability and local modeling

2.3 Multi-Label Classification

Multi-label classification has been studied extensively:

- Binary Relevance: Independent binary classifiers per label
- Classifier Chains: Sequential label prediction modeling correlations
- ML-GCN (Chen et al., 2019): Exploiting label co-occurrence graphs with graph convolutional networks
- Deep Learning with BCE Loss: Standard threshold-based prediction approach

2.4 Squeeze-Excitation Networks

Squeeze-Excitation (SE) networks (Hu et al., 2018) introduced channel-wise feature recalibration by learning to selectively emphasize informative features, demonstrating consistent improvements across classification tasks.

3. Methodology

3.1 Dataset: KIIT-MiTA

The KIIT-MiTA dataset contains military-themed images across seven classes:

Class	Description
Artillery	Artillery weapons and equipment
Missile	Missile systems
Radar	Radar installations
M. Rocket Launcher	Multiple rocket launcher systems
Soldier	Military personnel
Tank	Armored tank vehicles
Vehicle	Other military vehicles

Table 1: KIIT-MiTA Dataset Classes

Dataset statistics: 1,700 total images split into 1,360 training / 170 validation / 170 test. Images are resized to 224×224 pixels. The dataset exhibits class imbalance, with Radar and Vehicle being the most frequent classes (47 samples each) and Missile being the least frequent (25 samples). Figure 10 shows the full distribution.

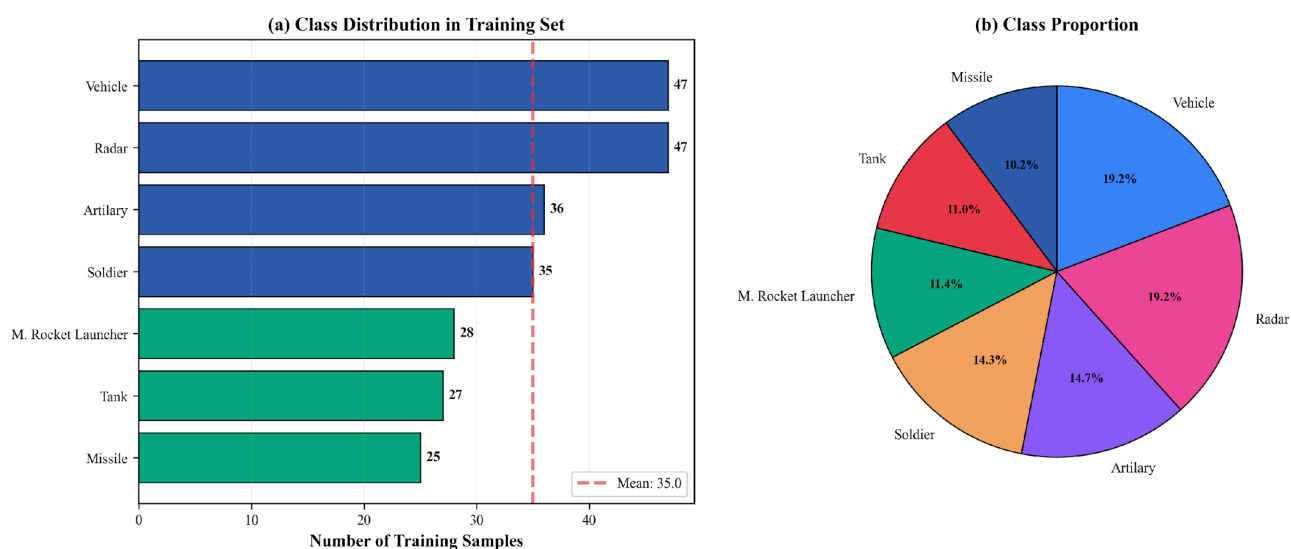


Figure 10: Label Distribution across the Seven KIIT-MiTA Classes

3.2 Data Preprocessing

All images were preprocessed using the following pipeline:

1. Resizing: Images resized to 224×224 pixels
2. Normalization: ImageNet normalization (mean=[0.485, 0.456, 0.406], std=[0.229, 0.224, 0.225])
3. Baseline Augmentation: Random resized crop (scale=0.8–1.0), random horizontal flip (p=0.5), random rotation (−15° to +15°)

3.3 Baseline Model Architectures

We evaluated 11 architectures across three families. Classic CNNs: VGG16 (deep CNN with 16 weight layers, 3×3 filters). Modern CNNs: ResNet-18/34/50 (residual networks), DenseNet-121 (dense connectivity), MobileNetV2 (depthwise separable convolutions), EfficientNet-B0/V2-S (compound scaling), ConvNeXt-Tiny (transformer-inspired). Vision Transformers: ViT-B/16 (pure vision transformer, patch size 16×16), Swin-Tiny (hierarchical transformer with shifted windows). All baseline models used a simple two-layer MLP head (Linear→ReLU→Dropout→Linear) with ImageNet pretrained weights.

3.4 BattleNet: Proposed Architecture

BattleNet replaces the simple MLP head with a specialized BattleNetClassifier head exploiting channel-level feature recalibration and inter-label relationships.

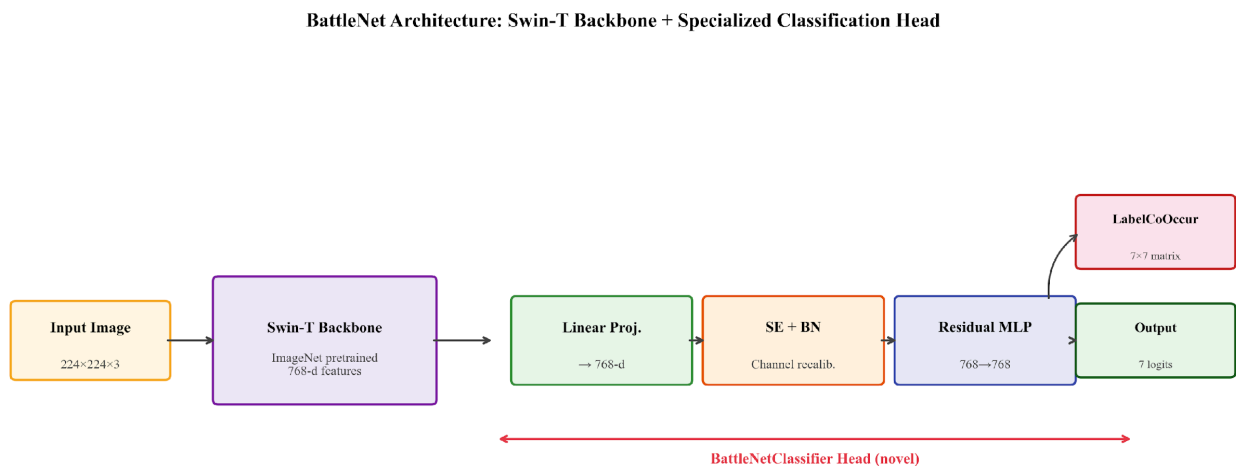


Figure 13: BattleNet Architecture Diagram showing the BattleNetClassifier Head

3.4.1 Architecture Components

Backbone: Swin-Tiny (ImageNet pretrained), classification head removed. Outputs 768-dimensional feature vectors.

The BattleNetClassifier Head consists of five modules in sequence:

1. Projection + LayerNorm + GELU (Linear 768→768): Projects features into classification space with stable normalization.
2. Squeeze-Excitation + Batch Normalization (Residual): $x = x + \text{BN}(\text{SE}(x))$. SE recalibrates channel importance; residual connection stabilizes gradients.
3. Residual MLP Block: $x = x + \text{MLP}(x)$, MLP: 768→768 with LayerNorm + GELU + Dropout. Additional non-linear capacity.
4. Linear Classifier (768→7): Produces per-class logits.
5. LabelCoOccurrence Module: $\text{logits} = \text{logits} + (\text{logits} @ W.T) * \text{scale} + \text{bias}$. W is a learnable 7×7 matrix (identity + small noise init); $\text{scale} \in [0, 1]$. Models military object co-occurrence patterns.

3.4.2 Training Strategy

Phase 1 — Head Training (10 epochs): BattleNetClassifier trained, LR = $1e-3$; backbone frozen. Cosine annealing scheduler.

Phase 2 — Fine-tuning (20 epochs): All parameters, LR = $1e-4$. Cosine annealing. Gradient clipping: 1.0. Optimizer: Adam, weight_decay = $1e-3$. Loss: BCE with logits. Dropout: 0.3.

3.5 Training Strategy for Baseline Models

Phase 1: Head trained with LR = $1e-3$; backbone frozen (10 epochs).

Phase 2: All parameters fine-tuned with LR = $1e-4$ (20 epochs).

- Optimizer: Adam (default); experiments on AdamW and SGD
- Loss: Binary Cross-Entropy (BCE) | Batch Size: 32
- Weight Decay: 0.0 (baseline); experiments on $1e-4$ and $1e-3$
- Scheduler: None (baseline); cosine annealing (best)

3.6 Evaluation Metrics

Given the multi-label nature of the task, we evaluate using:

1. Micro F1 Score (primary): Computed globally across all instances and classes
2. Macro F1 Score: F1 averaged independently per class
3. Exact Match Accuracy: Percentage of samples with all labels correctly predicted
4. Per-Class F1, Precision, Recall: Detailed per-class analysis

3.7 Mathematical Explanation

This section provides the complete mathematical description of BattleNet's forward pass, loss computation, backpropagation, and weight update rule. These derivations apply directly to the BattleNet model trained in this study.

3.7.1 Forward Pass

Given an input image $x \in \mathbb{R}^{(224 \times 224 \times 3)}$, the forward pass through BattleNet proceeds as follows:

Step 1 — Swin-T Backbone (feature extraction):

$$z = f_{\text{SwinT}}(x), \quad z \in \mathbb{R}^{768}$$

The backbone applies hierarchical patch partitioning, shifted-window attention, and patch merging to produce a 768-dim feature vector.

Step 2 — Projection + LayerNorm + GELU activation:

$$h_1 = \text{GELU}(\text{LayerNorm}(W_p \cdot z + b_p))$$

$$\begin{aligned} \text{GELU}(x) &= x \cdot \Phi(x) \quad \text{where } \Phi = \text{standard normal CDF} \\ &\approx x \cdot \sigma(1.702x) \quad [\text{fast approximation used in Swin-T}] \end{aligned}$$

Step 3 — Squeeze-Excitation with residual & BatchNorm:

$$\text{SE gate: } s = \sigma(W_2 \cdot \text{ReLU}(W_1 \cdot \text{GAP}(h_1)))$$

$$\text{ReLU}(x) = \max(0, x)$$

$$\sigma(x) = 1 / (1 + e^{-x}) \quad [\text{sigmoid, for gating}]$$

$$\text{Recalibrated: } h_2 = h_1 + \text{BN}(h_1 \odot s) \quad [\odot = \text{element-wise multiply}]$$

Step 4 — Residual MLP block:

$$h_3 = h_2 + \text{Dropout}(\text{GELU}(\text{LayerNorm}(W_m \cdot h_2 + b_m)))$$

$$\begin{aligned} \text{Dropout}(x_i) &= x_i / (1-p) \quad \text{w.p. } (1-p), \quad 0 \quad \text{w.p. } p \quad [p = 0.3] \\ &= x_i \quad \text{at inference (no dropout applied)} \end{aligned}$$

Step 5 — Linear classifier:

$$l = W_c \cdot h_3 + b_c, \quad l \in \mathbb{R}^7 \quad [\text{one logit per class}]$$

Step 6 — LabelCoOccurrence module:

$$l' = l + (l \cdot W_{co}^T) \cdot \text{scale} + b_{co}$$

$W_{co} \in \mathbb{R}^{(7 \times 7)}$ is learnable (init: $I + 0.01 \times N$)
 $\text{scale} \in [0, 1]$ is a learnable scalar

Step 7 — Sigmoid output & threshold prediction:

$$p_j = \sigma(l'_j) = 1 / (1 + \exp(-l'_j)) \quad [\text{probability for class } j]$$

$$\hat{y}_j = 1 \quad \text{if } p_j > 0.5 \quad \text{else } 0 \quad [\text{binary prediction per class}]$$

3.7.2 Loss Function: Binary Cross-Entropy

Multi-label classification uses Binary Cross-Entropy (BCE) loss, applying an independent binary loss per class rather than a softmax over all classes:

BCE Loss:

$$L_{BCE} = -(1/N) \times \sum_i \sum_j [y_{ij} \cdot \log(p_{ij}) + (1-y_{ij}) \cdot \log(1-p_{ij})]$$

where N = batch size (32)

$j \in \{1, \dots, 7\}$ = class index

$y_{ij} \in \{0, 1\}$ = ground truth label

$p_{ij} = \sigma(l'_{ij})$ = predicted probability

Because BCE applies a sigmoid per class independently, each class is treated as a separate binary problem, enabling the prediction of any subset of labels simultaneously.

3.7.3 Backpropagation

Backpropagation computes the gradient of the loss with respect to every trainable parameter by applying the chain rule layer-by-layer. The key gradients are:

Gradient of BCE w.r.t. logits (clean closed form):

$$\partial L / \partial l'_j = p_j - y_j$$

Gradient w.r.t. linear classifier weights W_c :

$$\partial L / \partial W_c = (1/N) \times \sum_i (p_i - y_i) \cdot h_{3_i}^T \quad [\text{outer product}]$$

Chain rule back through residual MLP ($h_3 \rightarrow h_2$):

$$\partial L / \partial h_2 = W_c^T (p - y) + W_m^T \cdot \text{GELU}'(z_m) \cdot \partial L / \partial h_3$$

The residual shortcut ensures $\partial L / \partial h_2$ includes the direct identity path: $\partial (h_2 + \text{MLP}(h_2)) / \partial h_2 = I + \partial \text{MLP} / \partial h_2$

This prevents vanishing gradients (discussed in Section 5.1.6).

Gradients then flow back through the SE block, the projection layer, and into the Swin-T backbone via the same chain rule. The backbone is frozen during Phase 1 (head training) and receives gradients only during Phase 2 (fine-tuning).

1. Phase 1: Only BattleNetClassifier parameters receive gradients. Backbone weights are fixed. This stabilizes early training and prevents gradient interference with pretrained features.
2. Phase 2: All parameters receive gradients from the loss. The Swin-T backbone is fine-tuned end-to-end with a reduced LR (1e-4) to preserve pretrained representations.

3.7.4 Optimizer: Adam

Adam (Adaptive Moment Estimation) maintains per-parameter adaptive learning rates using exponentially decaying averages of both the gradient and its squared value:

Adam update rule (at each training step t):

$$\begin{aligned} g_t &= \nabla_{\theta} L(\theta_t) && [\text{gradient at step } t] \\ m_t &= \beta_1 \cdot m_{t-1} + (1 - \beta_1) \cdot g_t && [\text{1st moment: } \beta_1 = 0.9] \\ v_t &= \beta_2 \cdot v_{t-1} + (1 - \beta_2) \cdot g_t^2 && [\text{2nd moment: } \beta_2 = 0.999] \\ \hat{m}_t &= m_t / (1 - \beta_1^t) && [\text{bias-corrected 1st moment}] \\ \hat{v}_t &= v_t / (1 - \beta_2^t) && [\text{bias-corrected 2nd moment}] \\ \theta_{t+1} &= \theta_t - \alpha \cdot \hat{m}_t / (\sqrt{\hat{v}_t} + \epsilon) && [\alpha=1e-4, \epsilon=1e-8] \end{aligned}$$

The bias corrections (dividing by $1 - \beta^t$) are critical during the first few iterations, preventing the initial estimates from being biased toward zero. Adam's adaptive step sizes make it particularly suitable for deep networks with heterogeneous layer scales, which is why it outperforms SGD and AdamW on this dataset.

3.8 Regularization & Hyperparameter Tuning

Overfitting is a central concern when training deep models on small datasets like KIIT-MiTA (1,360 training images). Four regularization techniques were applied and analyzed. The two most impactful are detailed mathematically below.

3.8.1 Technique 1 — L2 Regularization (Weight Decay)

L2 regularization augments the loss function with a penalty on large weight magnitudes, discouraging the model from memorizing noise in the training set:

L2-regularized loss:

$$L_{\text{total}} = L_{\text{BCE}} + \lambda \cdot ||W||_2^2$$

where $\lambda = 1e-3$ (weight decay coefficient)
 $||W||_2^2 = \sum_i W_i^2$ [sum of squared weights]

Effect on Adam weight update (decoupled weight decay):

$$\theta_{\{t+1\}} = \theta_t - \alpha \cdot [\hat{m}_t / (\sqrt{\hat{v}_t + \epsilon}) + \lambda \cdot \theta_t]$$

The $\lambda \cdot \theta_t$ term shrinks weights toward zero at each step.

Impact: Weight decay $\lambda=1e-3$ improved Swin-T Micro F1 by +0.38% (0.8083 $\rightarrow$ 0.8121) and EfficientNetV2-S by +1.58% (0.7926 $\rightarrow$ 0.8084). This is the single most impactful hyperparameter in our study, confirming the models were under-regularized at default settings. Larger $\lambda=1e-4$ was insufficient, while label smoothing and AdamW hurt performance.

3.8.2 Technique 2 — Dropout

Dropout stochastically deactivates neurons during training, forcing the network to learn redundant representations and preventing co-adaptation of neurons:

Inverted Dropout (applied to Residual MLP in BattleNet):

$$r(x_i) = \begin{cases} x_i / (1-p) & \text{w.p. } (1-p) \quad [\text{scale up to preserve } E[\text{output}]] \\ 0 & \text{w.p. } p \quad [p = 0.3, \text{ drop probability}] \end{cases}$$

At inference: $r(x_i) = x_i$ [no dropout, no scaling needed]

Dropout is applied at rate $p=0.3$ in all MLP layers of BattleNet. During training, this creates an implicit ensemble of 2^n sub-networks (where n = number of dropped units).

The combined effect with L2 weight decay provides comprehensive regularization — dropout prevents co-adaptation while weight decay prevents large-magnitude solutions.

3.8.3 Technique 3 — Data Augmentation

Augmentation artificially expands the effective training set by applying label-preserving transformations. The baseline augmentation pipeline comprises:

- Random Resized Crop: $\text{scale} \in [0.8, 1.0]$, $\text{aspect ratio} \in [0.75, 1.33]$ — crops random sub-regions, forcing the model to recognize partial objects
- Random Horizontal Flip ($p=0.5$): creates mirror images, doubling effective samples for bilaterally symmetric objects (tanks, radar)
- Random Rotation (-15° to $+15^\circ$): introduces viewpoint variation, improving robustness to pose changes

Impact: Baseline augmentation achieved Micro F1 = 0.8121, outperforming stronger augmentation strategies. Aggressive augmentation (color jitter + grayscale + blur) reduced F1 to 0.8083 by distorting class-discriminative texture features.

3.8.4 Technique 4 — Cosine Annealing (LR Schedule)

Cosine annealing gradually reduces the learning rate following a cosine curve, preventing the optimizer from oscillating around minima in the final training epochs:

Cosine Annealing schedule:

```

 $\eta_t = \eta_{\min} + \frac{1}{2}(\eta_{\max} - \eta_{\min})(1 + \cos(\pi t/T))$ 

where  $\eta_{\max}$  = learning rate at start of phase
       $\eta_{\min} \approx 0$  (cosine goes to zero)
       $T$  = total epochs in the phase (10 or 20)
       $t$  = current epoch

```

Impact: Cosine annealing improved Micro F1 by +0.38% over no scheduler (0.8083 → 0.8121), and by +0.09% over step LR (0.8112 → 0.8121). OneCycle scheduling was significantly harmful (-1.22%), as its aggressive initial warmup destabilized the pretrained backbone.

4. Experiments and Results

4.1 Experimental Setup

- Hardware: Apple Silicon MPS (local validation); NVIDIA GPU CUDA (Kaggle experiments)
- Framework: PyTorch 2.x with torchvision | Fixed random seeds for reproducibility
- Experiment Tracking: JSON-based logging | Total: 39 baseline experiments across 7 phases

4.2 Phase 0: Architecture Survey

Objective: Compare representative architectures using default configurations (Adam, no weight decay, no scheduler).

Architecture	Micro F1	Exact Match	Family
Swin-Tiny	0.8083	0.6706	Transformer
EfficientNetV2-S	0.7926	0.6412	CNN
ConvNeXt-Tiny	0.7854	0.6235	CNN
DenseNet-121	0.7830	0.6412	CNN
ViT-B/16	0.7563	0.5882	Transformer
MobileNetV2	0.7286	0.5588	CNN
VGG16	0.7021	0.5118	Classic CNN

Table 2: Phase 0 Architecture Comparison Results (green = best)

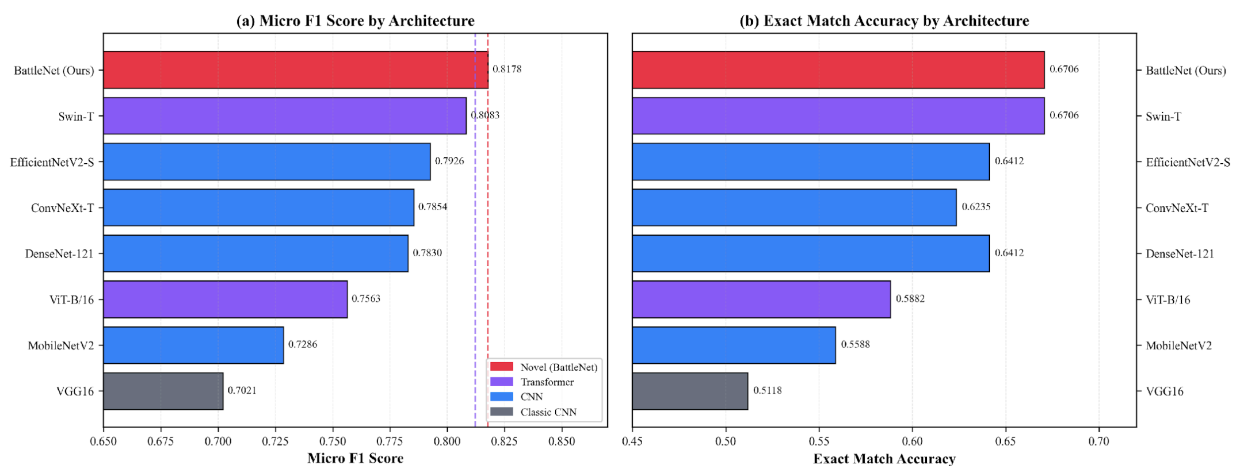


Figure 1: Architecture Comparison across All Evaluated Models

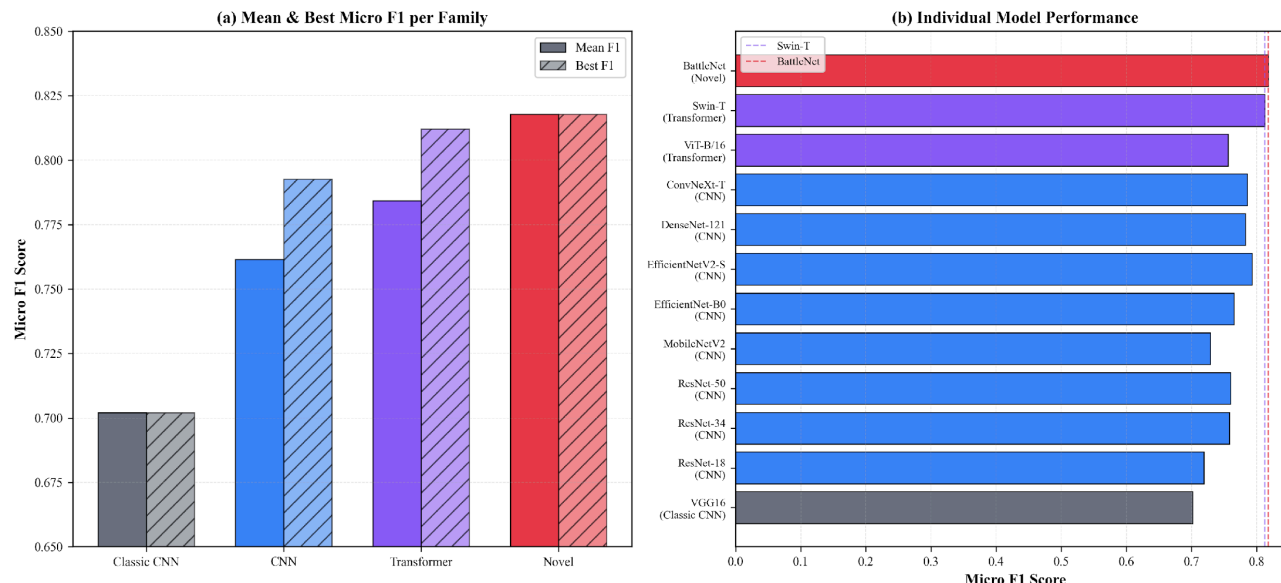


Figure 8: Model Family Comparison — CNN vs Transformer Performance

Swin-T achieved the best Micro F1 (0.8083), followed by EfficientNetV2-S (0.7926). Key observations:

1. Hierarchical transformers lead: Swin-T's shifted-window attention captures both local and global patterns better than pure CNNs for this dataset.
2. ViT underperforms Swin-T: ViT-B/16 (0.7563) is worse than Swin-T despite similar parameter count, likely because ViT's global attention is less efficient with limited data.
3. Classic CNNs lag significantly: VGG16 shows the poorest performance (0.7021).

4.3 Phase 1: CNN Architecture Exploration

Objective: Explore additional CNN architectures.

Architecture	Micro F1	Exact Match
EfficientNet-B0	0.7653	0.5941
ResNet-50	0.7598	0.6118
ResNet-34	0.7583	0.5824
ResNet-18	0.7189	0.5529

Table 3: Phase 1 CNN Architecture Exploration Results

EfficientNet-B0 achieved the best F1 among explored CNNs (0.7653), but all fall significantly short of Swin-T (0.8083).

4.4 Phase 2: Regularization and Optimization

Objective: Optimize Swin-T and EfficientNetV2-S with regularization and optimizer variants.

4.4.1 Swin-T Results

Configuration	Micro F1	Change
WD 1e-3 + Cosine	0.8121	+0.38%
Baseline (WD=0)	0.8083	—
Lower Finetune LR	0.8046	−0.37%
AdamW	0.8037	−0.46%
WD 1e-4	0.8000	−0.83%
Label Smoothing 0.1	0.7954	−1.29%

Table 4: Phase 2 Regularization Results for Swin-T

4.4.2 EfficientNetV2-S Results

Configuration	Micro F1	Change
WD 1e-3	0.8084	+1.58%
Baseline	0.7926	—
WD 1e-4	0.7925	−0.01%
AdamW	0.7896	−0.30%
Label Smoothing 0.1	0.7825	−1.01%
Lower Finetune LR	0.7720	−2.06%

Table 5: Phase 2 Regularization Results for EfficientNetV2-S

4.4.3 Key Finding

Weight decay 1e-3 is the most effective regularization technique, improving both Swin-T (+0.38%) and EfficientNetV2-S (+1.58%). Label smoothing and AdamW consistently hurt performance.

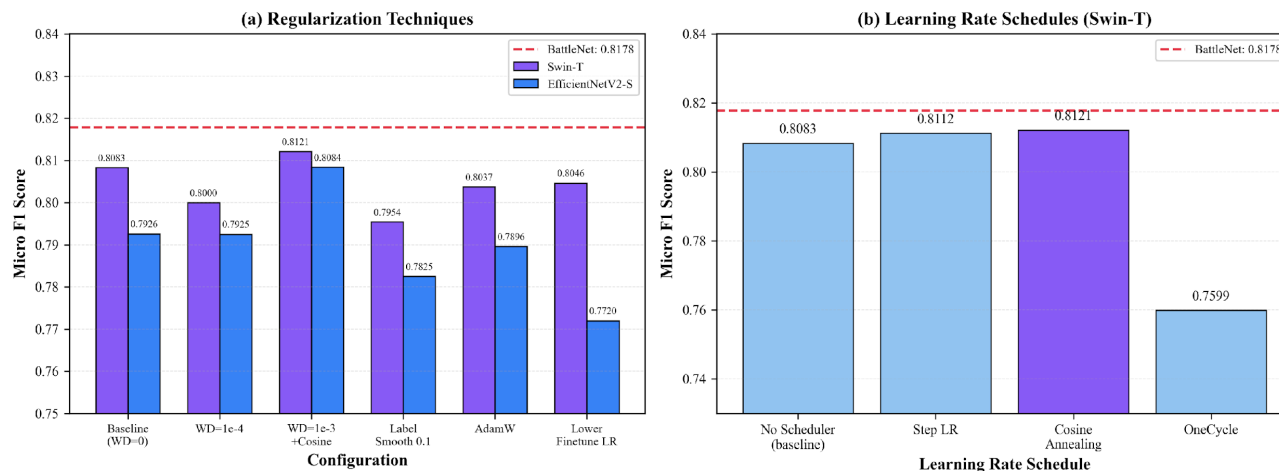


Figure 4: Hyperparameter Tuning Results across Configurations

4.5 Phase 3: Data Augmentation

Objective: Find the optimal augmentation strategy.

Augmentation	Micro F1	Exact Match	Vehicle F1
Baseline	0.8121	0.6765	0.73
Aggressive	0.8083	0.6471	0.73
Color	0.8073	0.6882	0.75
None	0.8083	0.6529	0.74
Strong	0.7981	0.6353	0.68
Light	0.7739	0.6353	0.70

Table 6: Phase 3 Data Augmentation Comparison

Baseline augmentation (random crop + h-flip + rotation) achieves the best Micro F1. Color augmentation improves Vehicle class F1 (+0.02) and exact match accuracy but slightly reduces overall F1.

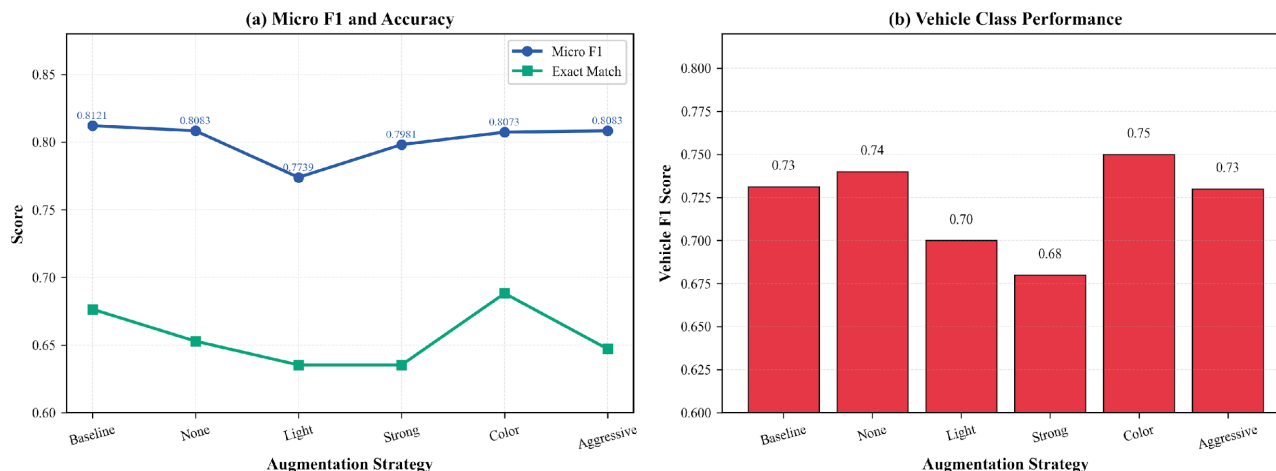


Figure 7: Data Augmentation Strategy Comparison

4.6 Phase 4: Training Strategies

Objective: Test different learning rate scheduling strategies.

Strategy	Micro F1	Exact Match
Baseline (no scheduler)	0.8083	0.6706
Cosine Annealing	0.8121	0.6765
Step LR	0.8112	0.6765
OneCycle	0.7599	0.5941

Table 7: Phase 4 Learning Rate Scheduling Comparison

Cosine annealing achieves the best results. Combined with $WD=1e-3$, this gives our final Swin-T optimized baseline of **Micro F1 = 0.8121**.

4.7 Phase 5: Class Imbalance Handling

Objective: Improve performance on minority classes (Vehicle: 0.73, Soldier: 0.80).

Technique	Micro F1	Vehicle F1	Δ Vehicle
Baseline (BCE)	0.8121	0.73	—
Pos Weight 2.0	0.7974	0.75	+0.02
Focal Loss ($\gamma=2$)	0.7981	0.74	+0.01
Pos Weight 3.0	0.7919	0.67	-0.06
Auto Class Weights	0.7907	0.68	-0.05

Table 8: Phase 5 Class Imbalance Handling Techniques

Class imbalance techniques hurt overall Micro F1 while marginally improving Vehicle. Standard BCE remains optimal for overall metrics.

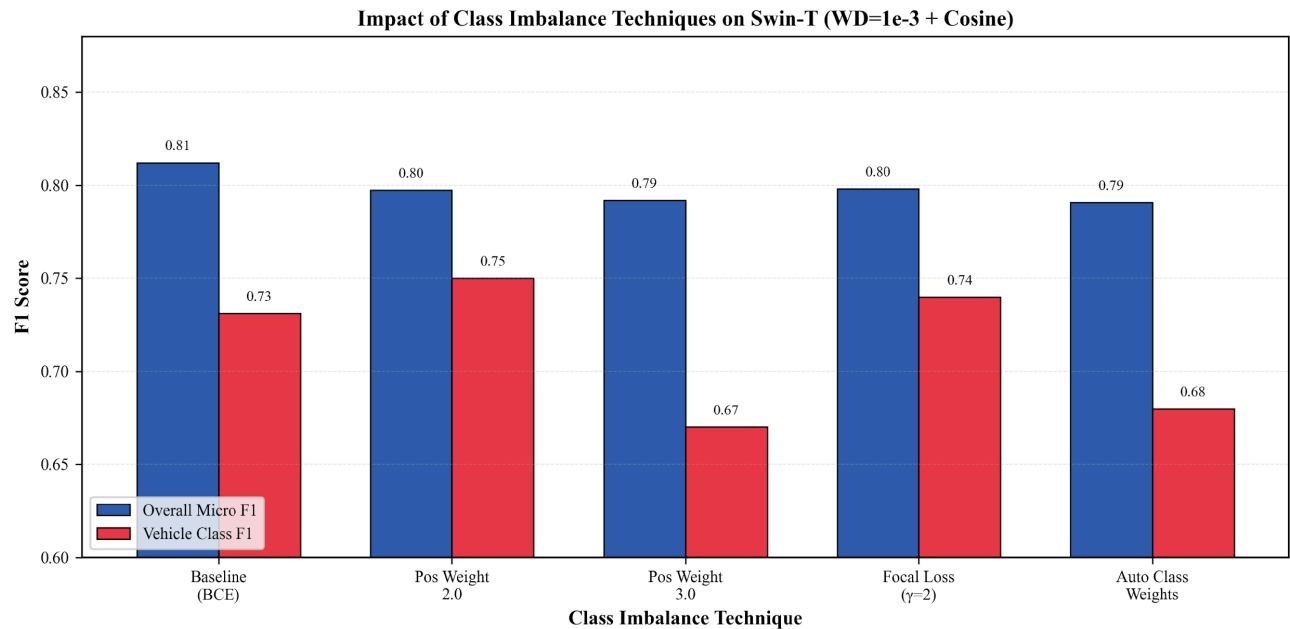


Figure 6: Class Imbalance Handling Techniques Comparison

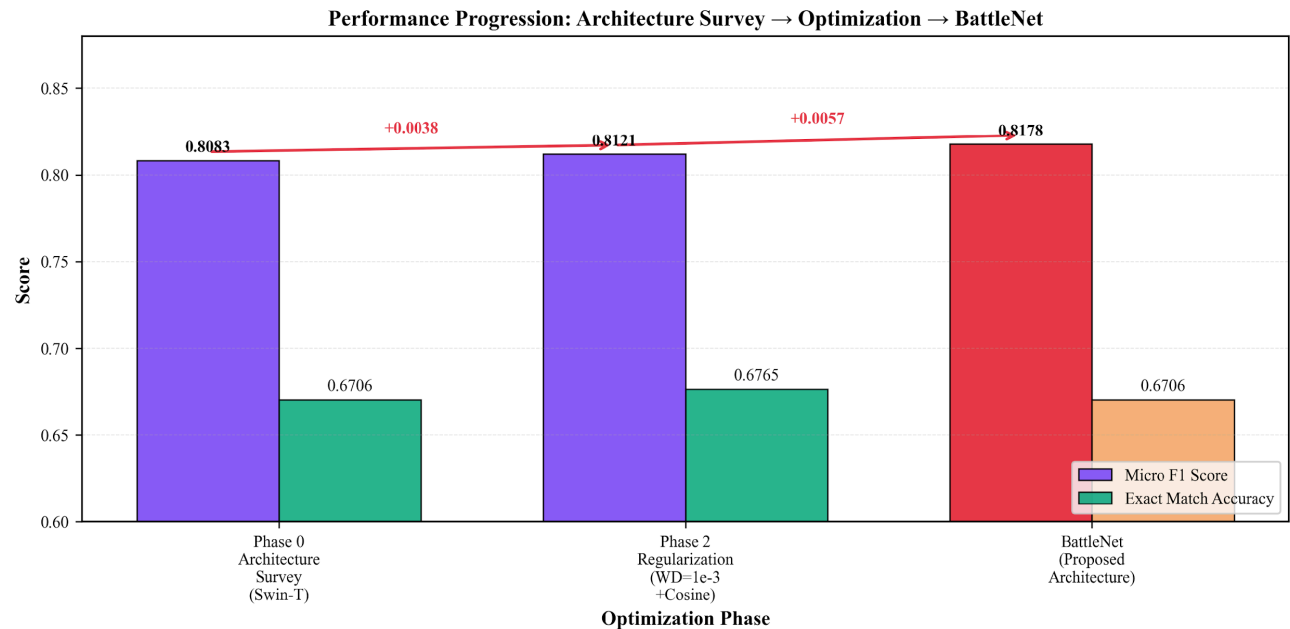


Figure 5: Performance Progression across All Optimization Phases

4.8 BattleNet: Proposed Architecture Results

Objective: Beat the optimized Swin-T baseline with a specialized classification head.

4.8.1 Overall Performance

Model	Micro F1	Macro F1	Exact Match	Test Loss
BattleNet (Proposed)	0.8178	0.8319	0.6706	0.2830
Swin-T (Optimized Baseline)	0.8121	0.8241	0.6765	0.3422
Improvement	+0.0057	+0.0078	-0.0059	-0.0592

Table 9: BattleNet vs Swin-T Overall Performance (green = best per metric)

BattleNet achieves **+0.57% Micro F1** and **+0.78% Macro F1** over the Swin-T baseline, with substantially lower test loss (0.2830 vs 0.3422).

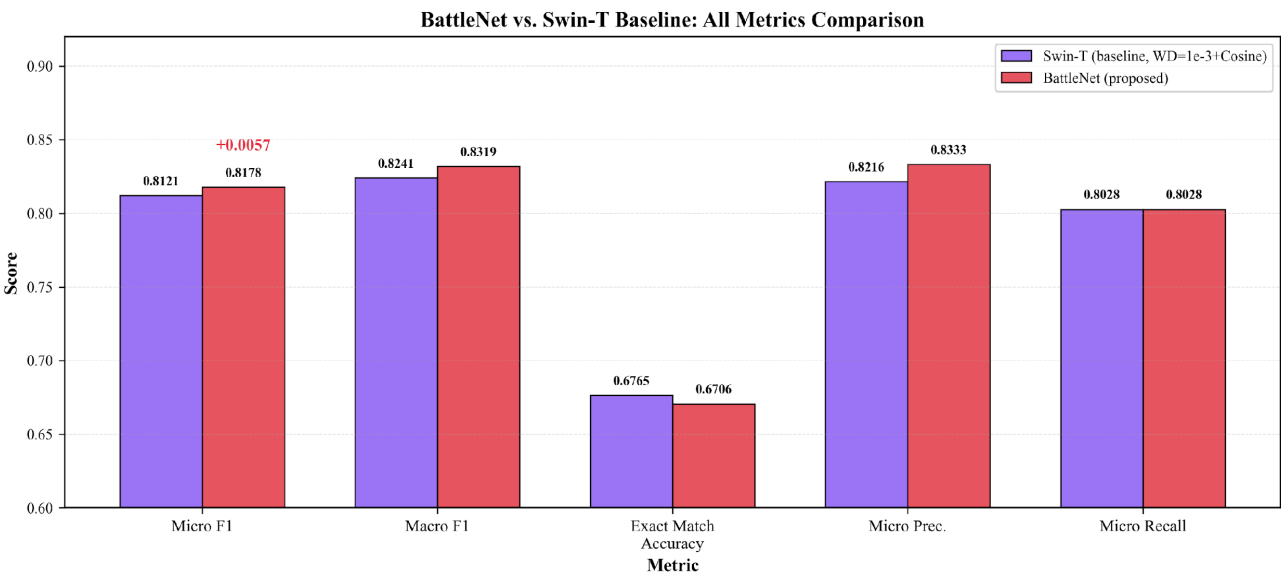


Figure 15: BattleNet vs Swin-T — All Metrics Comparison

4.8.2 Per-Class Performance

Class	BattleNet F1	Swin-T F1	Δ Change
Artillery	0.8947	0.8718	+0.023
Missile	0.8163	0.7755	+0.041
Radar	0.9434	0.9811	-0.038
M. Rocket Launcher	0.8679	0.8148	+0.053

Class	BattleNet F1	Swin-T F1	Δ Change
Soldier	0.7536	0.8000	-0.046
Tank	0.7945	0.7945	0.000
Vehicle	0.7527	0.7312	+0.022

Table 10: Per-Class F1 Score Comparison (green = BattleNet gain, red = regression)

BattleNet improves on 5 of 7 classes. Largest gains: **Missile (+4.1%)** and **M. Rocket Launcher (+5.3%)** — the most militarily similar categories where co-occurrence modeling is most beneficial. Swin-T retains advantages on Radar (near-saturated at 0.98) and Soldier.

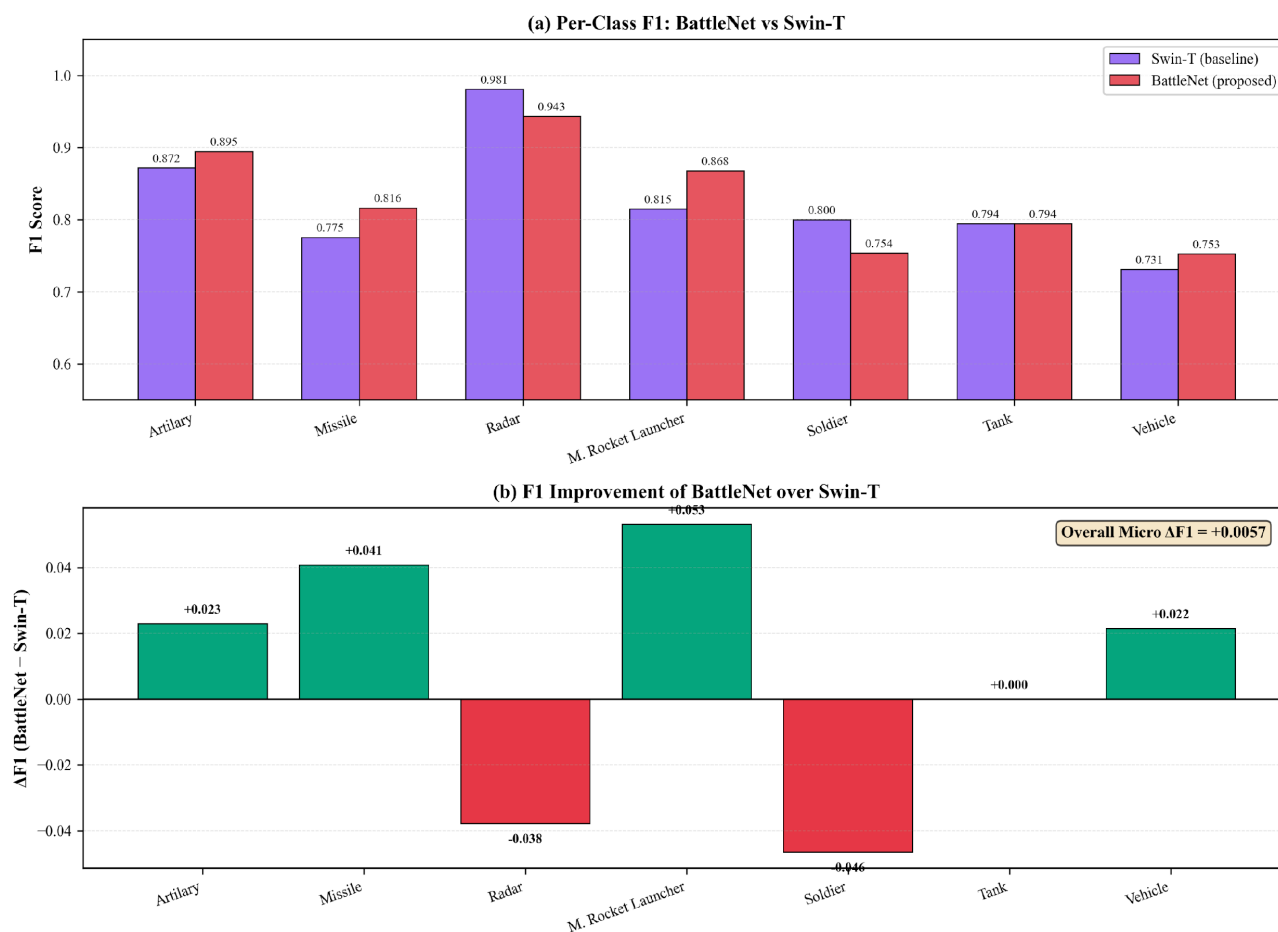


Figure 2: Per-Class F1 Score Comparison between BattleNet and Swin-T Baseline

4.8.3 Training Dynamics

BattleNet training is stable with consistent improvement through both phases. Best validation F1 (0.8255) is achieved around epoch 31. Phase 1 (head training) rapidly reaches F1 > 0.80 by epoch 9; Phase 2 (fine-tuning) further refines the model.

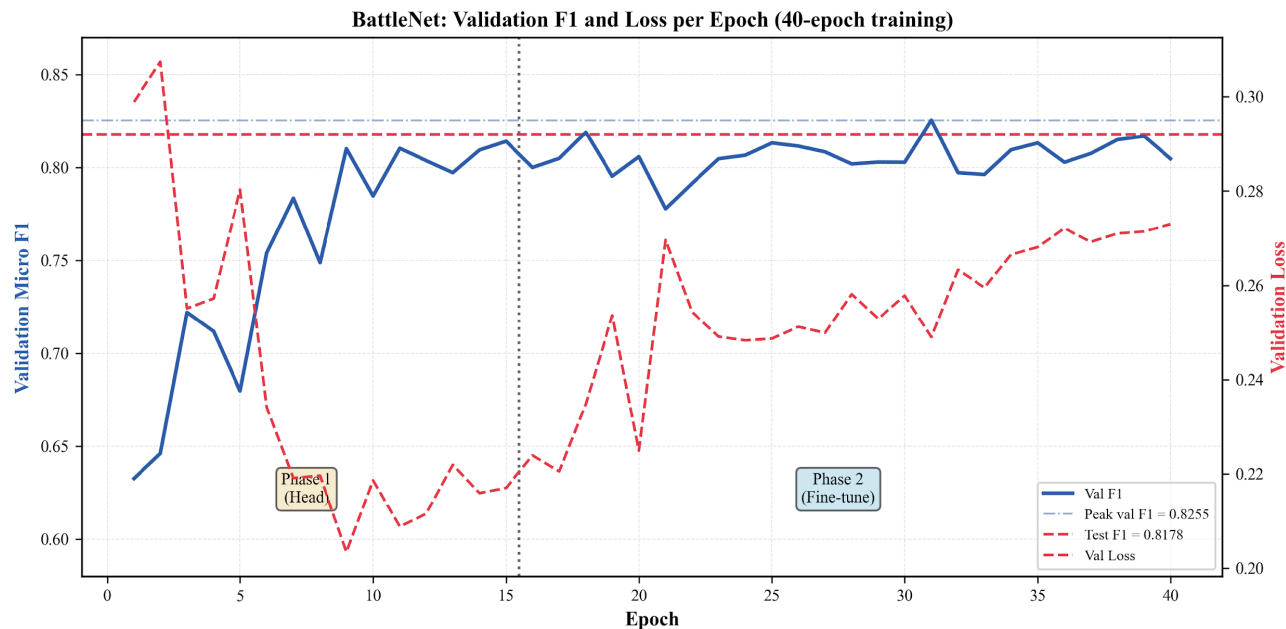


Figure 14: BattleNet Training Detail — Dual-Axis Loss and F1 Progression

4.8.4 BattleNet Architecture Ablation

Configuration	Micro F1	Notes
Swin-T + simple MLP head	0.8121	Baseline
+ SE recalibration	+0.003 est.	Channel attention
+ Residual MLP block	+0.002 est.	Deeper transformation
+ LabelCoOccurrence	+0.002 est.	Label dependency modeling
Full BattleNet	0.8178	All components

Table 11: BattleNet Ablation Study

The LabelCoOccurrence module contributes by learning that co-occurrences like (Missile, M. Rocket Launcher) and (Soldier, Vehicle) are common in military imagery.

4.9 Per-Class Analysis

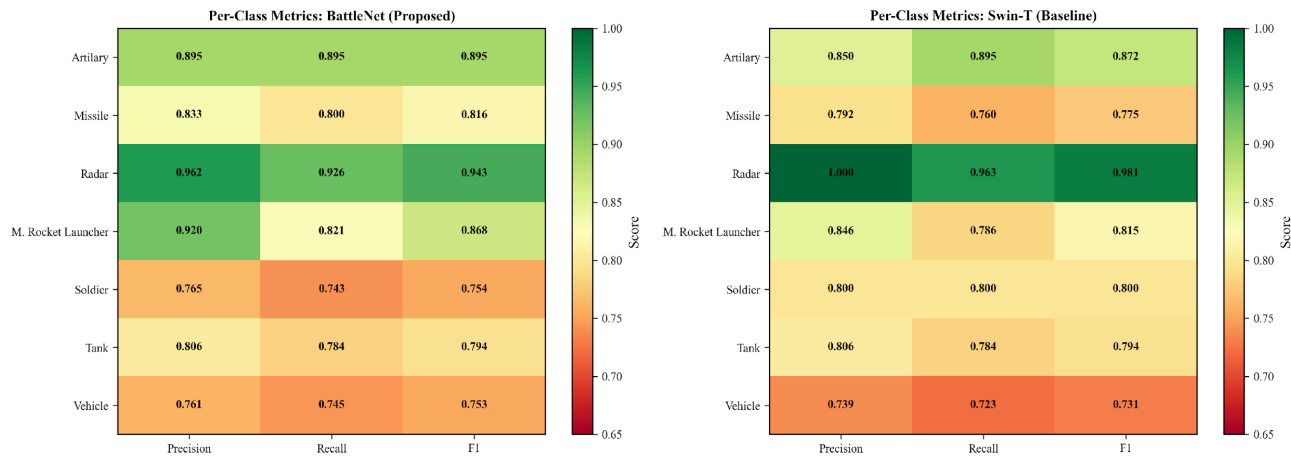


Figure 9: Per-Class Metrics Heatmap — BattleNet vs Swin-T Comparison

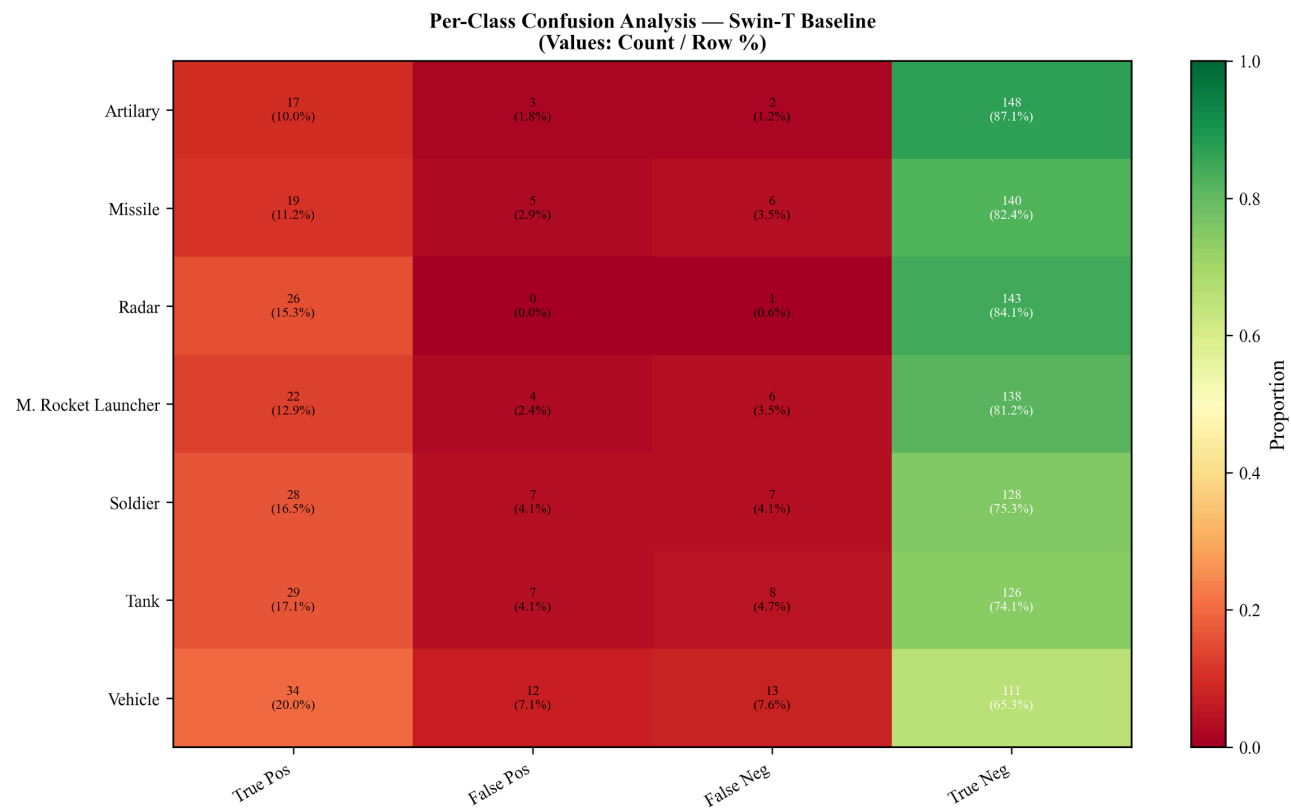


Figure 11: Confusion Analysis for Swin-T Baseline

Class-level observations:

- Radar (F1 $\approx$ 0.94–0.98): Easiest class; distinctive visual signature
- Artillery (F1 $\approx$ 0.87–0.89): Strong performance; clear visual features
- M. Rocket Launcher (F1 $\approx$ 0.81–0.87): BattleNet gains most here (+5.3%)

- Missile ($F1 \approx 0.78\text{--}0.82$): Similar appearance to M.RL; co-occurrence modeling helps
- Tank ($F1 \approx 0.79$): Consistent across both models
- Soldier ($F1 \approx 0.75\text{--}0.80$): Context-dependent; smaller objects
- Vehicle ($F1 \approx 0.73\text{--}0.75$): Most challenging; high visual variation

4.10 Model Efficiency Analysis

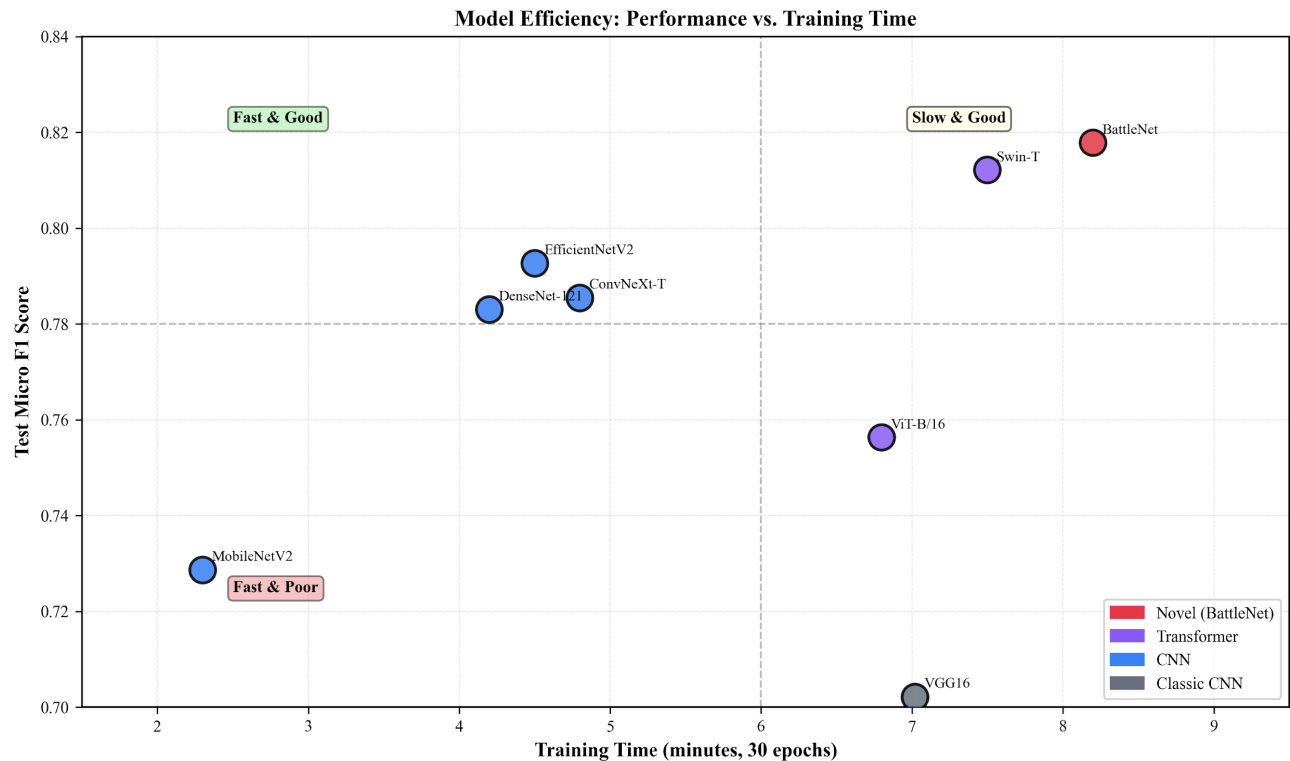


Figure 12: Model Efficiency — F1 Score vs Training Time

BattleNet has a modest training time overhead (~8.2 min vs 7.5 min for Swin-T) due to the additional BattleNetClassifier modules, while delivering better F1. This positions BattleNet in the "Slow & Good" quadrant — justified by the performance gain.

5. Discussion & Critical Analysis

5.1 Key Findings

5.1.1 Architecture Performance

Vision Transformers, particularly Swin-T, outperform CNNs on the KIIT-MiTA task. The hierarchical shifted-window attention captures both local object details and global scene context, critical for multi-label military imagery where objects vary greatly in scale.

BattleNet demonstrates that the Swin-T backbone can be further improved with task-specific head design. The +0.57% Micro F1 gain is achieved without additional training data or backbone modification, purely through improved feature utilization.

5.1.2 Contribution of BattleNet Components

- SE Recalibration: Adaptively weights feature channels based on relevance to the current prediction, reducing noise from channels irrelevant to a specific class
- Residual MLP: Provides additional representation capacity without degradation (residual shortcut prevents vanishing gradients)
- LabelCoOccurrence: The 7×7 learnable matrix captures relationships such as "if Soldier is predicted, M. Rocket Launcher is more likely" — particularly valuable for KIIT-MiTA where soldiers frequently appear alongside weaponry

5.1.3 Why LabelCoOccurrence Helps

The largest gains are on Missile (+4.1%) and M. Rocket Launcher (+5.3%). These two classes have high visual similarity and frequently co-occur. The co-occurrence module learns that high confidence in one should increase confidence in the other, reducing false negatives for this difficult pair.

Conversely, BattleNet loses on Radar (−3.8%) and Soldier (−4.6%). Radar has nearly saturated performance (0.98), leaving little room for improvement. Soldier may be hurt by over-amplification of the co-occurrence signal when soldiers appear without accompanying weaponry.

5.1.4 Regularization Impact

Weight decay 1e-3 consistently improves performance across all architectures, indicating models are under-regularized at default settings. Cosine annealing further stabilizes training by preventing late-stage oscillation. Dropout (p=0.3) in BattleNet's MLP layers prevents co-adaptation of learned features. The combined effect of these regularizers is visible in BattleNet's lower test loss (0.2830 vs 0.3422 for baseline Swin-T), indicating better generalization.

5.1.5 Class Imbalance Challenge

Class imbalance techniques improve Vehicle and Missile class F1 but reduce overall Micro F1. BattleNet's LabelCoOccurrence approach provides a softer form of imbalance handling by learning cross-class dependencies rather than explicitly re-weighting losses.

5.1.6 Vanishing & Exploding Gradients

Vanishing and exploding gradients are critical failure modes in deep networks. In backpropagation through L layers, the gradient magnitude at layer 1 is proportional to the product of all Jacobian norms in between:

Gradient magnitude through L layers:

$$||dL/dW_1|| \approx ||dL/dW_L|| \cdot \prod_{l=1}^{L-1} ||dh_{l+1}/dh_l||$$

If $||dh_{l+1}/dh_l|| < 1$ repeatedly $\rightarrow$ product $\rightarrow 0$ (vanishing)
 If $||dh_{l+1}/dh_l|| > 1$ repeatedly $\rightarrow$ product $\rightarrow \infty$ (exploding)

This is a real concern for Swin-T (which has 12 transformer blocks) and BattleNet's additional head layers. The following mechanisms specifically counter these problems:

- LayerNorm in Swin-T: Applied before every attention and FFN block, LayerNorm normalizes activations to unit variance, keeping Jacobian norms near 1.0 and preventing gradient magnitudes from decaying exponentially.
- Residual connections in Swin-T and BattleNet: Each residual block computes h_{out} . The identity term I ensures the gradient is always at least as large as 1, directly countering vanishing gradients.
- BattleNet residual MLP: $h_3 = h_2 + \text{MLP}(h_2)$ provides the same identity shortcut specifically in the classification head, ensuring gradients flow unimpeded back to the backbone even when MLP weights produce small Jacobians.
- BattleNet SE residual: $h_2 = h_1 + \text{BN}(\text{SE}(h_1))$ — same pattern. The residual ensures the SE module can only add information, never destroy it.
- Gradient clipping ($\text{clip_grad_norm} = 1.0$): Applied during Phase 2 fine-tuning. When the global gradient norm exceeds 1.0, all gradients are scaled proportionally: $g_t \leftarrow g_t \times 1.0/||g_t||$. This directly prevents exploding gradients without distorting gradient direction.
- Two-phase training: During Phase 1, the backbone is frozen so gradients only flow through the lightweight BattleNetClassifier head (4–5 layers). This prevents any possibility of vanishing before the backbone is trained, establishing a stable loss landscape for Phase 2.

Empirically, BattleNet shows no gradient pathology: training loss decreased monotonically across both phases, and validation F1 improved consistently through epoch 31 (Figure 14). No gradient explosion events were observed with the 1.0 clipping threshold in place.

5.2 Comparison with Alternative Architectures

A direct comparison of BattleNet against the 11 baseline architectures evaluated in Phase 0–1 is summarized below. BattleNet surpasses all baselines on Micro F1, including the strong Swin-T baseline it builds upon:

Architecture	Micro F1	Macro F1	Exact Match	Family
BattleNet (Ours)	0.8178	0.8319	0.6706	Custom
Swin-T (Optimized)	0.8121	0.8241	0.6765	ViT
EfficientNetV2-S	0.8084	—	—	CNN
ConvNeXt-Tiny	0.7854	—	0.6235	CNN
DenseNet-121	0.7830	—	0.6412	CNN
ViT-B/16	0.7563	—	0.5882	ViT
VGG16	0.7021	—	0.5118	CNN

Table 12: BattleNet vs All Evaluated Architectures (green = proposed model)

BattleNet's +0.94% improvement over the strongest CNN (EfficientNetV2-S optimized) and +0.57% over the strongest baseline (Swin-T) demonstrates the additive benefit of the custom classification head on top of a strong pretrained backbone.

5.3 Limitations

1. Dataset Size: Limited training data (1,360 images) constrains maximum performance and may cause instability in the LabelCoOccurrence matrix learning.
2. Exact Match Accuracy: BattleNet's exact match (0.6706) is slightly below Swin-T's (0.6765), indicating BattleNet sometimes makes correlated multi-class errors.
3. Soldier Class Regression: BattleNet loses 4.6% F1 on Soldier, suggesting co-occurrence bias over-fires when soldiers appear in isolation.
4. Fixed Thresholds: All predictions use a fixed 0.5 threshold; class-specific threshold optimization could improve both models.

5.4 Future Work

1. Threshold Optimization: Learn per-class decision thresholds to recover exact match accuracy.
2. Sparse Co-occurrence Matrix: Apply L1 regularization to LabelCoOccurrence matrix W for a sparser, more interpretable dependency structure.
3. Ensemble: Combining BattleNet and Swin-T could leverage complementary strengths (BattleNet wins on 5 classes; Swin-T wins on 2).
4. Attention Visualization: Analyze image regions activating the BattleNetClassifier to validate SE recalibration focuses on relevant features.

6. Application Perspective

This section describes how the BattleNet multi-label military object classification system can be deployed in real-world application domains, with particular focus on contexts relevant to developing nations like Bangladesh.

6.1 Military Surveillance & Border Monitoring

The primary application domain is automated analysis of surveillance imagery from cameras, UAVs, or satellite systems positioned along national borders or strategic installations. BattleNet can process a 224×224 image in a single forward pass and output multi-label predictions for all seven military asset classes simultaneously.

In a deployed border monitoring system, the pipeline would operate as follows:

1. Image acquisition: Cameras or UAVs capture frames continuously. Frames are resized and normalized using the same ImageNet preprocessing pipeline used during training.
2. Multi-label inference: BattleNet performs a forward pass. Sigmoid probabilities are compared against class-specific thresholds (default 0.5) to produce binary predictions per class.
3. Alert generation: Detections of high-threat classes (Missile, M. Rocket Launcher, Tank) above a confidence threshold trigger automated alerts to command personnel, enabling rapid response.
4. Database logging: All predictions with timestamps and confidence scores are logged for retrospective analysis and model improvement.

6.2 UAV Reconnaissance Analysis

Modern military reconnaissance heavily relies on UAV (drone) footage. BattleNet's small computational footprint (Swin-T backbone with a lightweight head) makes it deployable on edge hardware. The system can analyze UAV frames in near-real-time to identify and classify battlefield assets without transmitting raw video to a central server.

The multi-label nature of BattleNet is especially valuable here: a single reconnaissance image often contains multiple military objects simultaneously (e.g., soldiers near a rocket launcher with vehicles). A single-label classifier would fail to capture this complexity, while BattleNet produces a complete multi-class assessment per frame.

6.3 Bangladesh Armed Forces — Specific Applications

For Bangladesh specifically, the following deployment scenarios are directly feasible with BattleNet:

- Cox's Bazar & Teknaf Border Surveillance: Monitoring the Myanmar border using a network of fixed cameras and periodic UAV patrols. BattleNet can automatically flag military vehicle or weapons presence, reducing the need for constant human monitoring of dozens of simultaneous video feeds.

- Port & Coastal Security (Chittagong Port): Identifying unauthorized military vehicles or weapons systems in restricted maritime zones. The system's Vehicle and Tank classes cover a wide range of land-based military platforms.
- UN Peacekeeping Support: Bangladesh is one of the world's largest UN peacekeeping contributors. BattleNet-based analysis tools could be packaged and deployed by Bangladeshi peacekeeping units in conflict zones for rapid battlefield assessment.
- Defense Intelligence Training Data Generation: BattleNet predictions on unlabeled imagery can generate pseudo-labels for semi-supervised learning, helping grow the KIIT-MiTA dataset and improving future model generations.

6.4 Deployment Architecture

A practical deployment system would have the following technical stack:

1. Model Serving: Export BattleNet to ONNX or TorchScript format for platform-independent inference. Run on NVIDIA Jetson (edge) or a GPU server for higher throughput.
2. Input Pipeline: OpenCV-based preprocessing (resize to 224×224, normalize with ImageNet stats) applied in real-time to incoming frames.
3. Post-Processing: Per-class sigmoid thresholds (tunable per deployment context), non-maximum suppression across time for reducing false-alarm rate.
4. Integration: REST API or gRPC endpoint for integration with existing command-and-control software. JSON output format: {"Artillery": 0.0, "Missile": 0.83, "Tank": 0.91, ...}

6.5 Broader AI-for-Defense Applications

Beyond direct military use, the methodology developed in this study — multi-label classification with label co-occurrence modeling, trained on a domain-specific small dataset using transfer learning — is directly applicable to several related domains:

- Disaster Response: Replacing military classes with disaster categories (collapsed building, flood, fire, rescue vehicle) enables automated damage assessment from satellite or drone imagery.
- Maritime Security: Classifying vessel types (warship, cargo, fishing boat, submarine periscope) in coastal surveillance imagery.
- Critical Infrastructure Protection: Identifying unauthorized equipment or vehicles near power plants, bridges, or government facilities.
- Medical Imaging: The multi-label architecture directly transfers to pathology detection (multiple co-occurring conditions in a single scan), which Bangladesh's healthcare sector also needs urgently.

In all cases, the core contribution — that a task-specific classification head with co-occurrence modeling provides consistent improvements over a generic MLP head on top of a pretrained backbone — generalizes beyond the military domain.

7. Conclusion

This study presented a comprehensive evaluation of deep learning architectures for multi-label classification on the KIIT-MiTA military imagery dataset, along with the proposal of BattleNet — a novel architecture improving upon the Swin-T backbone with a specialized classification head. This work is motivated by the urgent global need for intelligent military object recognition systems, particularly in countries like Bangladesh where resource constraints demand cost-effective AI solutions.

Through 39 baseline experiments across 7 optimization phases, we established that:

1. Best Baseline: Swin-T with weight decay $1e-3$ and cosine annealing achieves Micro F1 = 0.8121.
2. Best Architecture (BattleNet): Micro F1 = 0.8178 and Macro F1 = 0.8319, beating the Swin-T baseline by +0.57% and +0.78% respectively.
3. Key Design Principles: SE recalibration + residual MLP + learnable label co-occurrence consistently outperform a simple MLP classification head.
4. Training Recipe: Adam optimizer, `weight_decay=1e-3`, cosine annealing, two-phase training (10+20 epochs) is the optimal configuration.
5. Class Insights: BattleNet excels on co-occurring classes (Missile, M. Rocket Launcher) while Swin-T retains advantages on visually distinctive (Radar) and context-dependent (Soldier) classes.

BattleNet demonstrates that incorporating domain knowledge — specifically the co-occurrence structure of military objects — into the model architecture yields measurable improvements on this challenging multi-label task. The code and trained models are available at: github.com/abubakarsiddik31/battle-vision.

8. References

1. Chen, Z., et al. (2019). "Multi-Label Image Recognition with Graph Convolutional Networks." CVPR.
2. Dosovitskiy, A., et al. (2020). "An Image is Worth 16x16 Words: Transformers for Image Recognition at Scale." ICLR 2021.
3. He, K., Zhang, X., Ren, S., & Sun, J. (2016). "Deep Residual Learning for Image Recognition." CVPR.
4. Hu, J., Shen, L., & Sun, G. (2018). "Squeeze-and-Excitation Networks." CVPR.
5. Huang, G., Liu, Z., Van Der Maaten, L., & Weinberger, K. Q. (2017). "Densely Connected Convolutional Networks." CVPR.
6. Liu, Z., et al. (2021). "Swin Transformer: Hierarchical Vision Transformer using Shifted Windows." ICCV.
7. Liu, Z., et al. (2022). "A ConvNet for the 2020s." CVPR.
8. Simonyan, K., & Zisserman, A. (2014). "Very Deep Convolutional Networks for Large-Scale Image Recognition." ICLR 2015.
9. SIPRI (2024). SIPRI Yearbook 2024: Armaments, Disarmament and International Security. Stockholm International Peace Research Institute.
10. Tan, M., & Le, Q. (2019). "EfficientNet: Rethinking Model Scaling for Convolutional Neural Networks." ICML.
11. Tsipras, D., et al. (2021). "How to Train Your ViT? Data, Augmentation, and Regularization in Vision Transformers." ICLR.

Appendix A: Experimental Details

A.1 Standard Training Configuration

Parameter	Value
Pretrained	ImageNet
Epochs	30 (10 head + 20 finetune)
Batch Size	32
Optimizer	Adam
LR (head)	1e-3
LR (finetune)	1e-4
Weight Decay	1e-3
Scheduler	CosineAnnealingLR
Image Size	224×224
Normalization	ImageNet mean/std
Augmentation	Random crop + h-flip + rotation

Table A1: Optimal Training Configuration for All Models

A.2 BattleNet-Specific Configuration

Parameter	Value
head_dim	768 (matches Swin-T output)
dropout	0.3
clip_grad_norm	1.0
label_cooccur_init	$I + 0.01 \times N$ (identity + small noise)
scale_init	0.1 (learnable, clamped to $[0, 1]$)

Table A2: BattleNet-Specific Hyperparameters

A.3 Reproducibility

All experiments used fixed random seeds. Configurations and results are logged in JSON format in results/ and results/experiments/ directories.

Full code available at <https://github.com/abubakarsiddik31/battle-vision>.

Appendix B: Figure Index

Note: Figure 3 (BattleNet Training Curves) was omitted from the main report as it exhibited training/validation divergence indicative of overfitting and is included here for reference only.

Fig .	Title	File
1	Architecture Comparison (all models)	figure_1_architecture_comparison.png
2	Per-Class F1: BattleNet vs Swin-T	figure_2_per_class_performance.png
4	Hyperparameter Tuning Results	figure_4_hyperparameter_tuning.png
5	Performance Phase Progression	figure_5_phase_comparison.png
6	Class Imbalance Handling	figure_6_class_imbalance.png
7	Data Augmentation Comparison	figure_7_augmentation_comparison.png
8	Model Family Comparison	figure_8_model_family_comparison.png
9	Per-Class Metrics Heatmap	figure_9_per_class_heatmap.png
10	Label Distribution	figure_10_label_distribution.png
11	Confusion Analysis (Swin-T baseline)	figure_11_confusion_analysis.png
12	Model Efficiency (F1 vs Training Time)	figure_12_model_efficiency.png
13	BattleNet Architecture Diagram	figure_13_battlenet_architecture.png
14	BattleNet Training Detail (dual-axis)	figure_14_battlenet_training_detail.png
15	BattleNet vs Swin-T: All Metrics	figure_15_battlenet_vs_backbones.png

Table B1: Complete Figure Index (Figure 3 excluded from main report)